Open Access | https://doi.org/10.59350/5fe9q-8ps48

Constitutive Models in Symbolic Form

Louis Moresi[1] [1] Australian National University

Abstract

A constitutive model is a Python class where the relationship between fluxes and gradients is encoded in SymPy. At every stage the mathematics is visible, inspectable, and differentiable. The framework handles Jacobians, C code generation, and PETSc integration. You handle the physics.

Keywords Underworld Code, Tricks of the Trade, development

In Underworld2, adding a new rheology was a matter of writing C code inside the StGermain framework, compiling it, and registering it with the component system. The barrier was high enough that most users never tried. The available rheologies were the ones the developers had implemented, and combining them required understanding the C internals.

In Underworld3, a constitutive model is a Python class where the relationship between fluxes and gradients is encoded as a SymPy expression. You can build a viscous model, add plasticity, add elasticity, make it anisotropic. At every stage the mathematics is visible, inspectable, and differentiable. The framework handles Jacobians, C code generation, and PETSc integration. You handle the physics.

```
stokes = uw.systems.Stokes(mesh)
stokes.constitutive_model = uw.constitutive_models.ViscousFlowModel
stokes.constitutive_model.Parameters.shear_viscosity_0 = viscosity_fn
```

This post explains how constitutive models work in UW3, from simple viscous flow through to viscoelastic-plastic rheologies with stress history.

Published Apr 13, 2026

Article UWTN 2026-006
Version 1.0.0

Live article
https://www.underworldcode.org/constitutive-models-in-symbolic-form/

1. The Constitutive Relationship

A constitutive model in Underworld3 defines the relationship between a flux (e.g. stress - a momentum flux) and gradients of the unknowns (e.g. strain rate - gradients of velocity). For a Stokes flow problem, the solver needs a flux term $\boldsymbol{F_1}$ that expresses the deviatoric stress:

$$\sigma_{ij} = C_{ijkl}, \dot{\varepsilon}_{kl} \tag{1}$$

where C_{ijkl} is the constitutive tensor (viscosity in this case) and $\dot{\varepsilon}$ is the symmetric strain rate tensor derived from the velocity gradient. For isotropic viscous flow, C_{ijkl} reduces to $2\eta, I_{ijkl}$ where η is the viscosity and I is the symmetric identity tensor. For more complex rheologies, the constitutive tensor can depend on the strain rate itself, on pressure, temperature, stress history, or material orientation.

The constitutive model's job is to build this tensor symbolically. The solver reads the model's `.flux` property, which returns the stress as a SymPy matrix expression. From there, the JIT pipeline described in our SymPy-to-C post takes over: automatically deriving Jacobians, unwrapping nested expressions, C code generation, PETSc integration.

2. Viscous Flow: The Starting Point

The simplest constitutive model is `ViscousFlowModel`. It has one parameter: shear viscosity.

```
stokes.constitutive_model = uw.constitutive_models.ViscousFlowModel
stokes.constitutive_model.Parameters.shear_viscosity_0 = uw.expression(
    r"\eta", uw.quantity(1e21, "Pa*s")
)
```

The viscosity can be a constant, a UWexpression with units, a SymPy expression involving temperature and pressure, or a mesh variable. The model does not care. It builds the stress tensor symbolically:

$$\sigma = 2\eta, \dot{\varepsilon} \tag{2}$$

You can inspect this at any time:

```
stokes.constitutive_model.flux
# Returns: 2 * η * ε(u)  — as a SymPy Matrix
```

In a Jupyter notebook, this renders as mathematics. You can see exactly what the solver will compute. If the viscosity expression is wrong, you see it here before running the solver.

3. Parameters as Guarded Descriptors

A common source of bugs in scientific code is mis-spelling a parameter name. You write `stokes.constitutive_model.Parameters.viscosty = 1e21` and nothing complains. The parameter you intended to set keeps its default value. The solver runs. The answer is wrong.

UW3's parameter system prevents this. Every constitutive model defines a `_Parameters` class whose attributes are descriptors. If you try to set an attribute that does not match a declared parameter, you get an immediate `AttributeError` listing the valid names:

```
stokes.constitutive_model.Parameters.viscosty = 1e21
# AttributeError: Cannot set 'viscosty' on ViscousFlowModel Parameters.
# Valid parameters: shear_viscosity_0
# (Did you mean 'shear_viscosity_0'? Use .viscosity as a shorthand.)
```

The descriptor names are the API. `shear_viscosity_0` is both the internal name and the user-facing setter. For convenience, viscous models also provide a `.viscosity` alias that maps to `shear_viscosity_0`.

Each parameter descriptor carries a LaTeX symbol, a default value factory, a description, and optional units. The defaults are created lazily through the owning model's symbol factory, ensuring that every parameter gets a unique SymPy symbol even when multiple models coexist.

4. Anisotropy and Tensor Representations

The scalar viscosity in `ViscousFlowModel` produces an isotropic constitutive tensor. But many geodynamics problems involve directional weakness: fault zones, shear bands, crystallographic fabric. `TransverseIsotropicFlowModel` handles this by introducing a director vector $\boldsymbol{n}$ and a second viscosity:

```
stokes.constitutive_model = uw.constitutive_models.TransverseIsotropicFlowModel
stokes.constitutive_model.Parameters.shear_viscosity_0 = eta_matrix   # matrix
viscosity
stokes.constitutive_model.Parameters.shear_viscosity_1 = eta_fault    # fault-
plane viscosity
stokes.constitutive_model.Parameters.director = n_vector              #
orientation
```

The constitutive tensor becomes:

$$C_{ijkl} = 2\eta_0, I_{ijkl} + 2(\eta_0 - \eta_1), A_{ijkl}(\boldsymbol{n}) \tag{3}$$

where A_{ijkl} is the anisotropic correction involving products of the director components. When $\eta_0 = \eta_1$, the correction vanishes and you recover isotropic flow. When $\eta_1 < \eta_0$, the material is weak along planes perpendicular to the director.

Building this tensor correctly requires care with index symmetries. The rank-4 constitutive tensor C_{ijkl} has 81 components in 3D (16 in 2D), but the symmetries of stress and strain rate reduce the independent entries. The standard approach in finite element work is to flatten the symmetric tensors into vectors and the constitutive tensor into a matrix. There are two common ways to do this, and the difference matters.

4.a. Voigt Notation

In Voigt notation, the stress and strain rate tensors are written as vectors by listing the independent components:

$$\tau_I = (\tau_{11}, \tau_{22}, \tau_{12}), \quad \dot{\varepsilon}_I = (\dot{\varepsilon}_{11}, \dot{\varepsilon}_{22}, 2\dot{\varepsilon}_{12}) \tag{4}$$

Note the factor of 2 on the off-diagonal strain rate. The constitutive matrix C_{IJ} is then the rearrangement of the rank-4 tensor without scaling. For isotropic viscosity in 2D:

$$\begin{bmatrix} \tau_{11} \\ \tau_{22} \\ \tau_{12} \end{bmatrix} = \begin{bmatrix} \eta & 0 & 0 \\ 0 & \eta & 0 \\ 0 & 0 & \eta/2 \end{bmatrix} \begin{bmatrix} \dot{\varepsilon}_{11} \\ \dot{\varepsilon}_{22} \\ 2\dot{\varepsilon}_{12} \end{bmatrix} \tag{5}$$

This is what you will find in most finite element textbooks. It works for computing stress from strain rate, but it has a problem: $\tau_I \dot{\varepsilon}_I \neq \tau_{ij} \dot{\varepsilon}_{ij}$. The vector inner product does not reproduce the tensor inner product. And C_{IJ} does not transform correctly under rotations.

4.b. Mandel Notation

Mandel notation fixes both problems by applying a scaling matrix $\boldsymbol{P}$ that puts a factor of $\sqrt{2}$ on the off-diagonal components:

$$

\tau^{ * } _ {I} = P _ {IJ},\tau _ {J}, \quad \dot\varepsilon^{ * } _ {I} = P _ {IJ},\dot\varepsilon _ {J}, \quad C^{ * } _ {IJ} = P _ {IK} ,C _ {KL} , P _ {LJ}

$$

where $\boldsymbol{P} = \mathrm{diag}\left(1, 1, \sqrt{2}\right)$ in 2D, or $\mathrm{diag}\left(1, 1, 1, \sqrt{2}, \sqrt{2}, \sqrt{2}\right)$ in 3D. In Mandel form, the isotropic constitutive matrix becomes:

$$

C^{ * } _ {IJ} = \eta , \delta _ {IJ}

$$

This is just η times the identity. The fourth-order symmetric identity tensor, which has an awkward $1/2$ factor in its off-diagonal rank-4 components, becomes the matrix identity in Mandel form.

The advantage of this approach is that rotations work naturally. If $\boldsymbol{R}$ is a rotation matrix, then the rotated Mandel constitutive matrix is:

$$

C'^{ * } _ {IJ} = R^{ * }_ {IK} , C^{ * } _ {KL} , R^{ * T} _ {LJ}

$$

where R^* is the Mandel-form rotation matrix derived from $\boldsymbol{R}$. This is why UW3 builds the transverse isotropic constitutive tensor in Mandel form. The anisotropic correction is defined in the material frame, rotated to the global frame using the director, and converted back to the rank-4 tensor. In Voigt notation, the same rotation would require tracking which components get the factor of 2 and which do not.

4.c. How UW3 Uses These Representations

The internal representation is the full rank-4 tensor C_{ijkl}. The Mandel form is available to the user through the `.C` property (capital C) for inspection and for supplying custom anisotropic tensors. The raw rank-4 tensor is available through `.c` (lowercase). If you provide a scalar viscosity, the model builds the rank-4 tensor directly. If you provide a Mandel matrix, the model converts it. Stress is passed to PETSc in Voigt form via `.flux_1d` to match its symmetric tensor storage conventions. The conversions between

these representations are handled by utility functions in `maths/tensors.py`, and the index book keeping is automatic and dimension-independent.

5. Adding Plasticity

`ViscoPlasticFlowModel` extends `ViscousFlowModel` with a yield stress. When the deviatoric stress exceeds the yield stress, the effective viscosity drops to keep the stress at the yield surface:

```
stokes.constitutive_model = uw.constitutive_models.ViscoPlasticFlowModel
stokes.constitutive_model.Parameters.shear_viscosity_0 = eta
stokes.constitutive_model.Parameters.yield_stress = uw.expression(
    r"\tau_y", uw.quantity(100, "MPa")
)
```

The plastic viscosity is computed from the yield stress and the second invariant of the strain rate:

$$\eta_{\mathrm{pl}} = \frac{\tau_y}{2,\dot{\varepsilon}_{II}} \tag{6}$$

The effective viscosity is the lesser of the viscous and plastic values.

$$\eta_{\mathrm{eff}} = \min\left(\eta_{\mathrm{v}}, \eta_{\mathrm{pl}}\right) \tag{7}$$

The model provides several other ways to combine them, because the choice affects Newton solver convergence. The default (“smooth”) form uses a corrected harmonic blend:

$$\eta_{\mathrm{eff}} = \eta_v \cdot \frac{1+f}{1+f+f^2}, \quad f = \frac{\eta_{\mathrm{v}}}{\eta_{\mathrm{pl}}} \tag{8}$$

This function is smooth everywhere, approaches η_v when $f \to 0$ (below yield), and approaches η_{pl} exactly when $f \to \infty$ (fully yielded). Other modes include harmonic averaging, a soft-min approximation, and a sharp min. The smooth default works well with Newton iteration because the Jacobian is continuous.

None of this blending logic requires special solver code. The effective viscosity is a SymPy expression. The solver differentiates it symbolically for the Jacobian. If you switch from smooth to sharp yielding, the Jacobian updates automatically.

6. Adding Elasticity: Stress Has Memory

Viscous and plastic models are instantaneous. The stress depends only on the current strain rate. Elastic behaviour introduces memory: the stress depends on the deformation history.

`ViscoElasticPlasticFlowModel` handles this. The Maxwell viscoelastic rheology combines viscous and elastic responses:

$$\dot{\varepsilon} = \frac{\sigma}{2\eta} + \frac{\dot{\sigma}}{2\mu} \tag{9}$$

Rearranging and discretising in time, the stress at the current step depends on the stress at previous steps. This stress history is a transported term, advected (and rotated) with the flow.

```
stokes = uw.systems.VE_Stokes(mesh, order=2)
stokes.constitutive_model = uw.constitutive_models.ViscoElasticPlasticFlowModel
stokes.constitutive_model.Parameters.shear_viscosity_0 = eta
stokes.constitutive_model.Parameters.shear_modulus = uw.expression(
    r"\mu", uw.quantity(1e10, "Pa")
)
```

The time discretisation uses backward differentiation formulas (BDF) with coefficients that adapt to variable timestep sizes. At order 1, this is the implicit Euler method. At order 2, BDF-2 gives second-order accuracy in time. When the timestep changes abruptly, the model falls back to BDF-1 automatically to avoid instabilities from extrapolating stress history over a large time gap.

The stress history lives on particles via the solver's `DFDt` (flux time derivative) infrastructure. When you assign a constitutive model that requires stress history, the solver creates the necessary particle storage and sets up advection automatically. The same BDF/Adams-Moulton framework that handles temperature advection handles stress advection. The constitutive model declares `requires_stress_history = True`, and the solver takes care of the rest.

If you don't want to use particles for tracking the stress history, you can use a semi-Lagrangian version of the `DFDt` which is a drop-in replacement at the user level.

For VEP problems, the viscoelastic effective strain rate includes contributions from the stress history, and the plastic yield criterion is evaluated against this total deformation rate. The `bdf_blend` parameter controls blending between BDF-1 and BDF-2 near the yield surface, where pure BDF-2 can produce oscillations. The model auto-detects the appropriate blend: pure VE problems get full BDF-2 accuracy, while VEP problems get a stable near-optimal blend.

Recent work has extended the anisotropic model to `TransverseIsotropicVEPFlowModel`, combining directional weakness with viscoelastic stress memory and plastic yielding. The yield criterion is evaluated on the resolved shear stress on the fault plane, computed from the full stress tensor and the director orientation. In UW3, this is a class that inherits from the VEP model and overrides the stress computation with additional director terms. The Jacobian follows automatically. In UW2, it would have been extremely difficult to implement.

7. The Solver's View

From the solver's perspective, a constitutive model is just an object with a `.flux` property that returns a SymPy Matrix. The same object pattern is used for constitutive models for stokes flow, heat diffusion, Darcy flow … The solver does not know whether the flux comes from a constant viscosity, a temperature-dependent Frank-Kamenetskii law, a viscoelastic model with stress history, or an anisotropic fabric model. It differentiates the flux to get the Jacobian, compiles both to C, and registers them with PETSc.

The assignment pattern reflects this:

```
# Assign a class — solver instantiates with its own Unknowns
stokes.constitutive_model = uw.constitutive_models.ViscousFlowModel

# Or assign an instance you've already configured
model = uw.constitutive_models.ViscoElasticPlasticFlowModel(stokes.Unknowns,
order=2)
model.Parameters.shear_viscosity_0 = eta
model.Parameters.shear_modulus = mu
stokes.constitutive_model = model
```

When you assign a model, the solver shares its `Unknowns` object with the model. This gives the model access to the velocity gradient (for computing strain rate), the DFDt stress history (for viscoelasticity), and the coordinate system (for computing directors in curvilinear geometry, for example). The model and solver are collaborators, not independent objects.

8. The Design Pattern

The constitutive model system embodies a design choice that runs through all of Underworld3: separate the physics from the numerics. The physics lives in the constitutive model. It knows about viscosity, yield stress, elastic moduli, directors, stress history. It expresses all of this using SymPy objects.

The numerical part lives in the solver. This knows about weak forms, Jacobians, PETSc assembly, Newton iteration, time stepping. It reads the model's symbolic expressions and compiles them.

The boundary between the two is a SymPy Matrix. Everything on one side of that boundary is human-readable physics. On the other side is machine-generated numerics. You can change the physics without touching the solver. You can change the solver without touching the physics. And because the boundary is symbolic, both sides are inspectable at every stage.

In UW2, the physics and numerics were entangled in C. Changing one required understanding both. In UW3, you can write a new rheology in an afternoon, in a notebook, without compiling anything. That is the practical consequence of making constitutive models symbolic objects.